

Felhők hálózati szolgáltatásai laboratórium
(BMEVITMMB11)

Vorleser **OCR alkalmazás**

Kovács Bertalan
RHDBLM

2026. április 19.



Tartalomjegyzék

1. CI/CD környezet	– 3 –
1.1. Projekt struktúra	– 3 –
1.2. A verziókezelő rendszer struktúrájának elve	– 3 –
1.3. A Flask alkalmazás bemutatása	– 4 –
1.4. Az OKD erőforrások létrehozása	– 5 –
1.5. Folyamatos integráció	– 6 –
1.6. Összegzés	– 8 –
Hivatkozások	– 10 –

1. CI/CD környezet

Ez a fejezet a Vorleser programhoz készült CI/CD környezet konfigurálását és beállítását taglalja egy egyszerű, két komponensből álló, [Python](#) alapú, [Flask](#) szolgáltatáson keresztül. A CI/CD rendszer a [Git](#) alapú [GitHub](#) szolgáltatást használja verziókezelő és integrációs, az [OKD](#)-t pedig telepítési és futtatási környezetnek. OKD implementációnak a [BME által futtatott](#) OKD rendszert választottam.

? Miért OKD?

Az OKD a natív Kubernetes fölé építve olyan fejlesztői (PaaS) funkciókat biztosít, mint az integrált CI/CD – amelynek köszönhetően a telepítési folyamatok könnyen triggerelhetők GitHubról, és a rendszer a forráskódból automatikusan le is buildeli a szükséges Docker image-eket –, valamint a beépített szoftveres útválasztás és a robusztus belső DNS alapú szolgáltatás-felfedezés. Ezek a funkciók kellenek igazán a lazán csatolt mikroszolgáltatások kommunikációjához.

1.1. Projekt struktúra

A példában egy kétszintű Flask alkalmazást hoztam létre. A projekt könyvtárszerkezete a következő:

```
vorleser/  
|-- backend/  
|   |-- app.py  
|   |-- requirements.txt  
|   |-- Dockerfile  
|-- frontend/  
|   |-- app.py  
|   |-- requirements.txt  
|   |-- Dockerfile  
|-- .github/workflows/OKD_deploy.yaml
```

A projekt megtalálható a GitHubon: <https://github.com/berci9ke101/vorleser>, ez a repozitórium fogja a későbbi Vorleser OCR program verziókezelő rendszerét biztosítani, tehát ide fognak majd a későbbi fájlok is kerülni.

1.2. A verziókezelő rendszer struktúrájának elve

A mikroszolgáltatások elveit követve minden komponens saját könyvtárral és Dockerfile-al rendelkezik, minimalizálva a függőségeket és így elősegítve a könnyű hordozhatóságot. A példában az alábbi Dockerfile-t használtam mindkét komponensnél (backend és frontend):

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

A későbbi OCR alkalmazás struktúrája is ezeken az alapokon fog működni.

1.3. A Flask alkalmazás bemutatása

A CI/CD pipeline bemutatásához egy egyszerű Flask alkalmazást hoztam létre, ami-
ben a backend publikál egy szövegrészletet a `/api/data` útvonalán. Ezt a szöveget
felhasználva a frontend megjeleníti a backend üzenetét. A forráskódok alább olvashatóak:

Backend:

```
from flask import Flask, jsonify

app = Flask(__name__)
@app.route('/api/data')
def get_data():
    return jsonify(message="Hello World!")

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Frontend:

```
from flask import Flask
import requests
import os

app = Flask(__name__)
@app.route('/')
def index():
    try:
        response = requests.get(f"http://{os.getenv('BACKEND_URL')}  
↪ :5000/api/data")
        data = response.json()
        return f"<h1>Backend says: {data['message']}</h1>"
    except Exception as e:
        return f"<h1>Error: {str(e)}</h1>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=3000)
```

A frontend a `BACKEND_URL` környezeti változón meghatározott néven keresztül éri el

a backendet. Ezt később be is állítottam.

1.4. Az OKD erőforrások létrehozása

Miután bejelentkeztem a BME webes felületén az OKD konzolba, kimásoltam a bejelentkezési parancsomat, majd az OKD platformon az alábbi parancsokkal inicializáltam a projektet és a szolgáltatásokat:

```
# Bejelentkezés az OKD parancssori alkalmazásába
oc login --token=<..> \
    --server=https://api.okd.fured.cloud.bme.hu:6443

# A projekt létrehozása
oc new-project vorleser

# A build-ek létrehozása GitHub forrásból, ügyelve a kontextusra
oc new-app https://github.com/berci9ke101/vorleser.git \
    --context-dir=/backend \
    --name=vorleser-backend \
    --strategy=docker

oc new-app https://github.com/berci9ke101/vorleser.git \
    --context-dir=/frontend \
    --name=vorleser-frontend \
    --strategy=docker

# Szolgáltatások helyezése a komponensek elé és a portjaik kinyitása
oc expose deployment/vorleser-backend --port=5000 --target-port=5000
oc expose deployment/vorleser-frontend --port=3000 --target-port=3000

# A frontend szolgáltatás kiengedése alapértelmezett tanúsítvánnyal
oc expose svc/vorleser-frontend
oc patch route vorleser-frontend \
    -p '{"spec":{"tls":{"termination":"edge"}}}'
```

Az OKD lehetővé teszi, hogy a verziókezelő rendszerben az egyes szolgáltatáskomponensek külön könyvtárban helyezkedjenek el. Ekkor az egyes komponensek kontextusát, a verziókezelő rendszerben elhelyezkedő, elérési útvonaluk alapján be kell állítani a fentebb látott `-context-dir=<..>` kapcsoló segítségével.

Az alkalmazás elérhetőségéhez szolgáltatásokat hoztam létre a komponensek elé és kiengedtem a frontendet a nyílt Internetre. Mivel a klaszter alapértelmezetten a HTTPS használatára kényszerít, így az alapértelmezett bejövő tanúsítványt kellett használnom, azaz be kellett állítani az *Edge TLS termination*-t [1].

A frontendnek megadtam a fentebb már említett backend elérhetőségét környezeti változóként:

```
oc set env deployment/vorleser-frontend BACKEND_URL=vorleser-backend
```

1.5. Folyamatos integráció

A folyamatos integrációt a GitHub Actions segítségével valósítottam meg. Minden `main` ágra történő push esemény automatikusan elindítja a kód szintaktikai ellenőrzését. Ahhoz, hogy az OKD értesüljön az új verzióról a GitHub action bejelentkezik a BME OKD szerverére és értesíti az új verzióról. Ezután az OKD elkezd a Docker image-ek buildelését.

Az autentikációhoz létre kell hozni egy `serviceaccount`-ot. Ennek a menetét az alábbi parancsokkal végeztem el¹:

```
cat <<EOF | oc apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: custom-build-instantiator
  namespace: vorleser
rules:
- apiGroups: ["build.openshift.io"]
  resources: ["buildconfigs/instantiate"]
  verbs: ["create"]
EOF

oc create rolebinding github-trigger-strict-binding \
  --role=custom-build-instantiator \
  --serviceaccount=vorleser:github-trigger \
  -n vorleser

oc create sa github-trigger
oc create token github-trigger --duration=8760h
```

A sikerességről a következő képek számolnak be:

```
berci9ke101@BINARY-DOLPHIN:~$ cat <<EOF | oc apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: custom-build-instantiator
  namespace: vorleser
rules:
- apiGroups: ["build.openshift.io"]
  resources: ["buildconfigs/instantiate"]
  verbs: ["create"]
EOF

oc create rolebinding github-trigger-strict-binding \
  --role=custom-build-instantiator \
  --serviceaccount=vorleser:github-trigger \
  -n vorleser
role.rbac.authorization.k8s.io/custom-build-instantiator created
rolebinding.rbac.authorization.k8s.io/github-trigger-strict-binding created
berci9ke101@BINARY-DOLPHIN:~$ |
```

1. ábra. A megfelelő role létrehozása.

¹A szükséges parancsok a BME OKD üzemeltetője, Bodor András útmutatása alapján születtek.

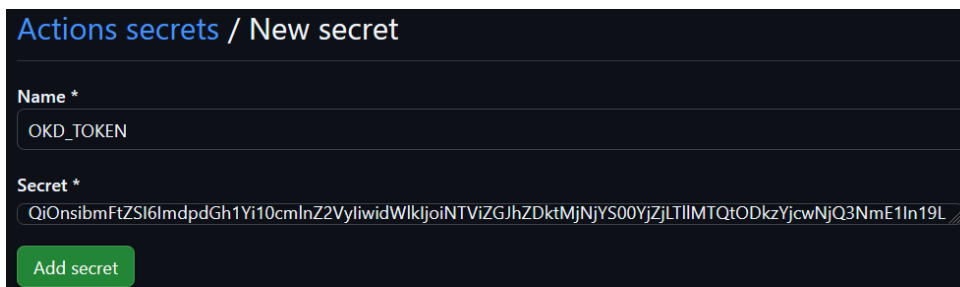
```

berci9ke101@BINARY-DOLPHIN:/mnt/c/Users/Bertalan/Nextcloud/Egyetem/_MSc/3/LABOR/vorleser$ oc create sa github-trigger
oc create token github-trigger --duration=8760h
serviceaccount/github-trigger created
eyJhbGciOiJSUzI1NiIsImtpZCI6IHRwbG9kRmRmNWpyUWwLLWdRcXVudT3NEWEN4Q1U1Zkg4M2c0YmQ1dDZkdSFUifQ.eyJhdwQiOiIlsiaHR0cHM6Ly9rdWw3

```

2. ábra. A token létrehozása a rolehoz.

A kapott tokenet el kell tárolni a GitHub repozitóriumban az **Action secrets** fül alatt:



3. ábra. A GitHub Actions token eltárolása.

Maga az action az alábbi yaml fájl segítségével fut le [2]:

```

name: Trigger OKD Build

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.9'

      - name: Install dependencies and Pylint
        run: |
          python -m pip install --upgrade pip
          pip install pylint
          pip install -r backend/requirements.txt
          pip install -r frontend/requirements.txt

      - name: Run Pylint
        run: |

```

```
pylint backend/app.py frontend/app.py

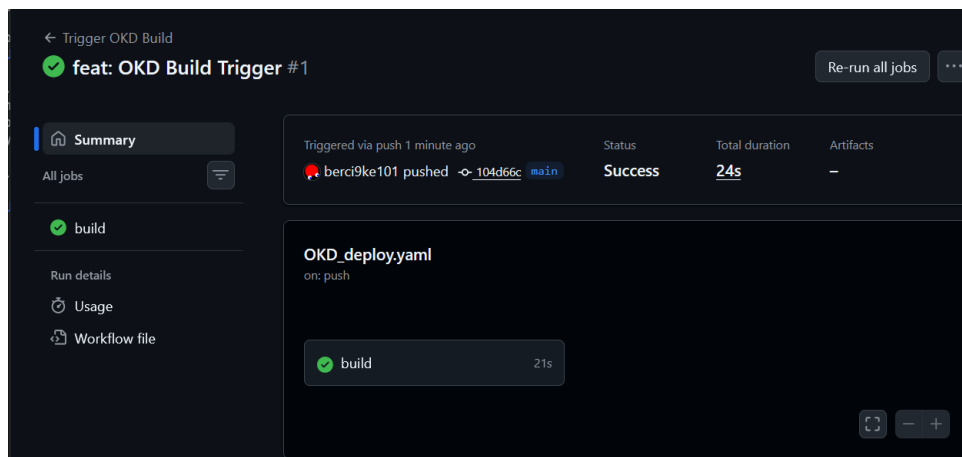
- name: Install oc CLI
  uses: redhat-actions/openshift-tools-installer@v1
  with:
    oc: 4

- name: Login to OKD
  uses: redhat-actions/oc-login@v1
  with:
    openshift_server_url:
      ↪ https://api.okd.fured.cloud.bme.hu:6443
    openshift_token: ${ secrets.OKD_TOKEN }
    namespace: vorleser

- name: Start Backend Build
  run: oc start-build vorleser-backend

- name: Start Frontend Build
  run: oc start-build vorleser-frontend
```

Az action sikeres lefutása után már nézhetjük is az OKD webes felületén a, ahogy a komponenseink elindulnak:

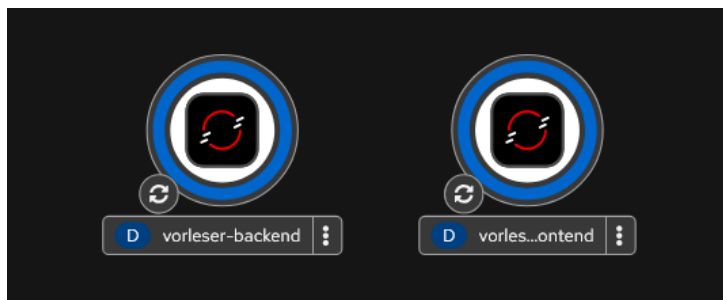


4. ábra. Sikeres GitHub Actions lefutás.

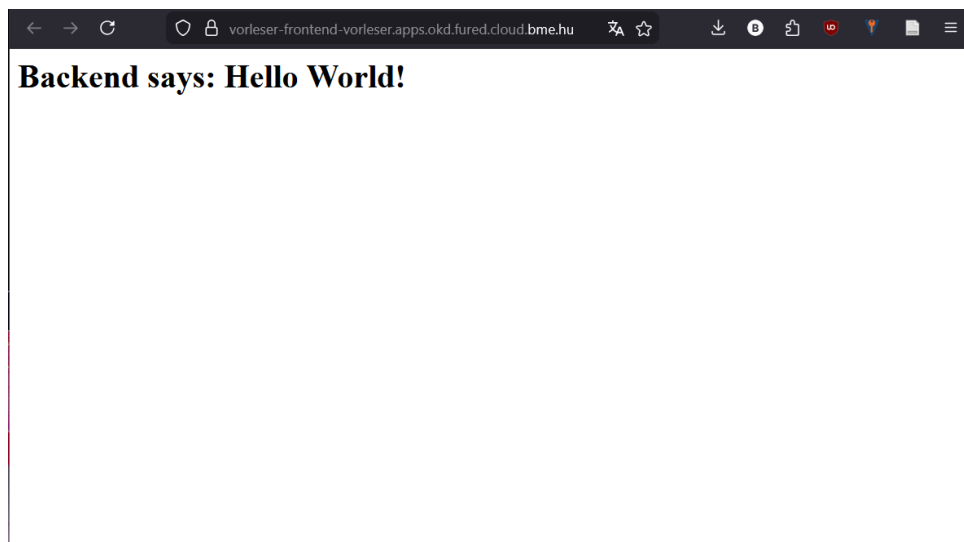
Megtekintve a <https://vorleser-frontend-vorleser.apps.okd.fured.cloud.bme.hu/> linket láthatjuk, hogy működik az alkalmazás:

1.6. Összegzés

A jelen fejezetben bemutatott folyamat elsősorban a Vorleser alkalmazás alapjául szolgáló CI/CD környezet és az OKD platform képességeinek demonstrálására fókuszált. A felépített architektúra egy leegyszerűsített, kétkomponensű Flask alkalmazáson keresztül igazolta a GitHub Actions és az OKD integrációjának zökkenőmentes működését.



5. ábra. Folyamatban lévő build-ek az OKD konzolon.



6. ábra. A működő alkalmazás.

Fontos megjegyezni, hogy ez az implementáció csupán egy kezdeti, „*proof of concept*” fázis volt az alapvető telepítési folyamatok hitelesítésére. A végleges optikai karakterfelismerő (OCR) mikroszolgáltatás-rendszer bevezetésekor ez a bemutatott alap egy jóval kiforrottabb, „kipolított” CI/CD pipeline-ná fog bővülni. A végleges folyamat a kódminőség szigorúbb ellenőrzését, komplexebb tesztelési fázisokat és a mikroszolgáltatások egymástól független, finomhangolt frissítését is automatizálni fogja a stabil felhős üzemeltetés érdekében.

2. Weboldal + OCR detektálás funkció

Ez a fejezet a Vorleser alkalmazás elosztott architektúráját és a fejlesztés során hozott technológiai döntéseket mutatja be. A rendszer egy mikroszolgáltatás-alapú ökoszisztémára épül, ahol az egyes funkciókat – mint a webes interfész (*Papagei*), az OCR feldolgozás (*Leseratte*) és a bináris adattárolás (*Maulwurf*) – önálló, saját fejlesztésű, egymástól lazán csatolt konténerizált komponensek látják el. A fejezet részletezi az aszinkron feladatkezelést biztosító *RabbitMQ* (*Briefstaube*) üzenetsor integrációját, a *EasyOCR* könyvtár alkalmazását, valamint a *PostgreSQL* (*Elefant*) alapú perzisztens adattárolást.

2.1. Projekt struktúra

A Vorleser alkalmazás jelenlegi struktúrája a következő (dokumentáció, licenszelés stb. elhagyásával):

```
vorleser/
|-- .github/workflows/OKD_deploy.yaml # CI/CD pipeline leíró
|-- k8s/ # Deklaratív kubernetes leírók
|   |-- blob-storage-deployment.yaml
|   |-- blob-storage-pvc.yaml
|   |-- frontend-deployment.yaml
|   |-- ocr-worker-deployment.yaml
|   |-- postgres-deployment.yaml
|   |-- postgres-pvc.yaml
|   |-- rabbitmq-deployment.yaml
|-- leseratte/ # OCR Worker
|   |-- app.py
|   |-- requirements.txt
|   |-- Dockerfile
|-- maulwurf/ # Blob Storage
|   |-- app.py
|   |-- requirements.txt
|   |-- Dockerfile
|   |-- test_app.py
|-- papagei/ # Frontend & API Gateway
|   |-- templates/
|   |   |-- 404.html
|   |   |-- index.html
|   |   |-- view.html
|   |-- app.py
|   |-- requirements.txt
|   |-- Dockerfile
|-- docker-compose.yaml # Lokális teszteléshez
```

2.2. Komponensek bemutatása és technológia választások indoklása

- **Papagei (Frontend & API Gateway):** A *Papagei* nem csupán a webes alkalmazás, hanem egy átjáró, amely összefogja a *Maulwurf* (tároló), az *Elefant* (adatbázis) és a *Brieftaube* (üzenetsor) hívásait. Konténerizáltam, hogy az OKD-ban egyszerűen publikálható és elérhető legyen kívülről.
- **Leseratte (OCR feldolgozó):** Az OCR művelet (EasyOCR modellek betöltése és futtatása) lassú és számításigényes. Ha ez a frontendben futna, a felhasználói felület megállna. Ha megnő a feltöltött képek száma, példányszáma növelhető anélkül, hogy a frontendhez hozzá kellene nyúlni.

? *Miért nem FaaS?*

Bár az OCR feladat elméletileg alkalmas lenne FaaS (Function as a Service) megoldásra a párhuzamos végrehajtás miatt, a konténeres megoldást választottam, mert az EasyOCR modellek több gigabájtosak és betöltésük jelentős időt vesz igénybe, ami egy FaaS filozófiába nem illene bele.

- **Maulwurf (Blob tároló):** Ahelyett, hogy minden pod közvetlenül csatlakozna egy megosztott fájlrendszerhez, a *Maulwurf* egy egységes API-t biztosít a szolgáltatásoknak a képek tárolására. Ez lehetővé teszi, hogy később a fájlrendszert akár S3-ra vagy más felhős tárolóra cseréljük. Egy PVC biztosítja a fájlok integritását egy újraindítást követően.
- **Elefant (PostgreSQL adatbázis):** A képek metaadatait (`leírás`, `állapot`, `blob_id`) relációs formában kell tárolni a konzisztencia miatt. Egy PVC biztosítja a fájlok integritását egy újraindítást követően.
- **Brieftaube (RabbitMQ üzenetsor):** A *Papagei* (Frontend & API Gateway) és a *Leseratte* (OCR feldolgozó) közötti laza csatolást a *Brieftaube* valósítja meg, amely garantálja a feladatok megőrzését a feldolgozó egység túlterheltsége vagy elérhetetlensége esetén is. Hiszen a megszakadt munkafolyamatok üzenetei automatikusan visszakérülnek a sorba az ismételt végrehajtáshoz.

Hivatkozások

- [1] OKD Community, *Creating advanced routes - Creating a route using the default certificate through an Ingress object*. Elérhető: https://docs.okd.io/4.16/networking/routes/creating-advanced-routes.html#nw-ingress-edge-route-default-certificate_creating-advanced-routes (Letöltve: 2026-03-15).
- [2] Red Hat, *OpenShift Login GitHub Action*. Elérhető: <https://github.com/redhat-actions/oc-login> (Letöltve: 2026-03-15).